SOLUTION REPORT

Grounded legal help and debate practice

A two-workflow legal assistant for everyday disputes in India.

Problem and choice

People facing routine disputes such as unreturned rental deposits, defective purchases, or unpaid wages need a clear first step but may lack easy access to legal guidance. We chose a focused assistant that structures the issue, retrieves relevant legal excerpts, explains options, and supports drafting or debate practice. A domain-bounded, evidence-first workflow is more useful than an unrestricted legal chatbot, while still requiring a lawyer for consequential decisions.

Scope and assumptions

Supported areas are rent and tenancy, consumer protection, and wages. A general fallback handles other topics without presenting them as verified domain advice. India is the default country context; state, forum, dates, contract terms, law commencement, and facts can change the result. Uploaded PDFs are context, not automatically authoritative law. The Streamlit text interface supports PDF upload and a separate Debate Practice mode. It provides information and drafting support, not representation, and is not yet multilingual or voice-enabled.

Evidence pipeline

PDF ingestion extracts selectable text and uses OCR where needed, then conservatively cleans and structure-chunks content. Persistent Chroma collections are separated by legal domain. Deterministic chunk IDs and retained source, page, section, and other metadata support traceability and targeted updates.

Multi-query retrieval: the original question plus three optional LLM rewrites broadens recall when everyday wording differs from statutory wording. Alternatives are checked for protected details such as section numbers, years, dates, amounts, and durations. Results merge by stable chunk ID to prevent repeats.

Hybrid search: BGE-M3 embeddings with Chroma find semantically similar passages; BM25 finds exact terms and statutory labels. LangChain EnsembleRetriever fuses ranked lists with reciprocal-rank fusion, default weights 0.5/0.5, to produce up to 20 candidates inside the selected domain.

Reranking: a Voyage cross-encoder ranks candidates against the original query and returns up to six. Cohere is tried if Voyage responds with HTTP 429; other errors surface. Reranking improves ordering but cannot establish that a passage is current or applicable.

Grounding: provenance follows each passage into citations. If no usable excerpts are found, the system should abstain from domain-specific claims. A matching citation ID does not prove the passage entails the claim.

Request path

Question -> safety check -> supervisor -> domain retrieval -> analysis or draft -> compliance and section checks -> response.

Two agents for two user goals

Normal legal-help agent

The LangGraph StateGraph records the turn, runs a local safety check, and asks a structured supervisor to classify the issue. It routes to rent, consumer, or wages retrieval agents, asks for clarification when routing confidence is low, and uses a cautious general fallback outside those domains. Domain nodes return structured analysis with sources. Drafting shapes a notice or response, followed by checks for citation IDs, urgency, limitation warnings, and unsupported statutory section numbers before the conversation node replies.

Debater agent

A separate LangGraph first clarifies the issue, jurisdiction, user position, and readiness to practise. After confirmation, it routes to rent, consumer, or wages, retrieves excerpts, and argues the opposing side with source labels. History supports turn-by-turn rebuttal; a coaching request switches to feedback on strengths, gaps, and one concrete improvement. If retrieval is empty, it declines to invent a law-based argument. This is practice, not adjudication or legal advice.

Framework, state, and LangSmith

The stack is Python, LangChain for model and retriever integrations, LangGraph for stateful routing, Chroma for persistent vectors, BM25 for lexical search, and Streamlit for the interface. Groq or Google models can be configured, and workflow dependencies are injectable for evaluation. The UI uses in-memory checkpoints for both graphs. The normal graph also supports PostgreSQL for durable thread history; debate persistence is in-memory by default.

Optional LangSmith tracing is wired into both streamed workflows. Set LANGSMITH_TRACING=true, an API key, and optionally endpoint/project settings to inspect graph runs, node paths, latency, and errors. Tracing is observability, not a quality verdict. Inputs may include sensitive narratives or uploaded-document text, so deployment should set trace access and retention deliberately; redaction is not assumed.

Evaluation and scaling

Repository test plans compare dense, BM25, and hybrid search; toggle expansion and reranking; and inspect collection isolation, duplicate IDs, provenance, and scores. A tenant deposit debate case checks clarification, routing, source-supported opposition, continuity, coaching, and abstention. These are plans, not measured accuracy. Quality review should use expert-reviewed prompts and sources to assess citation entailment, jurisdiction/date handling, draft fidelity, safe abstention, and urgent routing. Track retrieval quality, unsupported claims, abstentions, latency, and provider cost. Expand with reviewed, versioned sources, jurisdiction-aware routing, expert evaluation, retention limits, and human escalation.

Known limits and failure modes

This is an early prototype without independent legal validation. OCR can corrupt text; sources can be stale or incomplete; query rewrites can drift; rerankers can misorder passages; and fluent models can overstate the evidence. Current compliance validates citation IDs, flags urgency and limitation uncertainty, and corrects unsupported section numbers, but does not perform full entailment or jurisdiction audits. Consumer limitation handling is a date-based heuristic; rent and wage deadlines remain uncertain. Human review is needed before filing or sending a legal notice.

Repository scenarios are evaluation plans, not evidence of measured accuracy. For a full component map, see the architecture diagram in README.